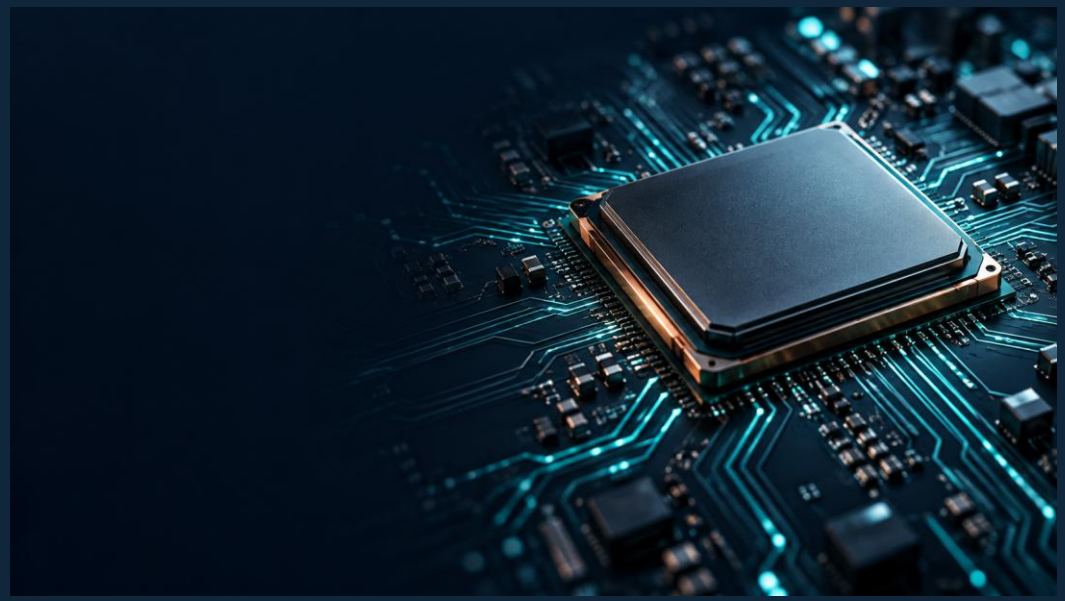
Máquina virtual simple

VM STUDIO

Memoria, registros y ciclo de instrucciones

Implementación didáctica en Python



Integrante 1 · Integrante 2 · Integrante 3

Problema y objetivo

Las instrucciones cambian el estado de la CPU,
pero esos cambios normalmente no son visibles.

Objetivo del proyecto

Interpretar programas pequeños y observar, después de cada instrucción, cómo cambian PC, IR, ACC y memoria.

Arquitectura del simulador

Componente	Responsabilidad
Interfaz y consola	Cargar, ejecutar y mostrar el mismo estado
Unidad de control	Coordinar Fetch, Decode y Execute
Memoria y registros	Conservar instrucciones, datos y estado
Controladores	Adaptar teclado, archivos y salida
Reportes	Conservar una instantánea de la ejecución

Un solo motor para las dos interfaces

Memoria e instrucciones

100 celdas direcciones 0–99

El código inicia en 0. DATA inicializa datos sin ocupar código.

Dirección	Contenido inicial	Tipo
0–4	LOAD, ADD, STORE, OUTPUT, HALT	Código
10	10	Dato
11	20	Dato
20	0 → 30 al ejecutar STORE	Dato

Registros de la máquina

Registro	Función	Tras ADD 11
PC	Dirección de la próxima instrucción	2
IR	Última instrucción obtenida	ADD 11
ACC	Dato de trabajo y resultado	30
Z / N	ACC es cero / ACC es negativo	0 / 0

PC es una dirección; IR es una instrucción; ACC es un dato.

Lenguaje de instrucciones

Grupo	Instrucciones	Efecto
Transferencia	LOAD · STORE	Leer y escribir memoria
Aritmética	ADD · SUB	Modificar ACC
Control del flujo	JMP · JZ · JNZ	Cambiar PC
Entrada y salida	INPUT · OUTPUT	Leer y emitir enteros
Terminación	HALT	Detener la ejecución

DATA dir valor es una directiva del cargador; no consume ciclos.

Ciclo Fetch Decode Execute

Fase	Operación	Ejemplo ADD 11
FETCH	Leer Mem[PC] y copiar a IR	IR = ADD 11
DECODE	Identificar opcode y operando	Operación ADD; dirección 11
EXECUTE	Aplicar el efecto de la operación	ACC = 10 + 20 = 30

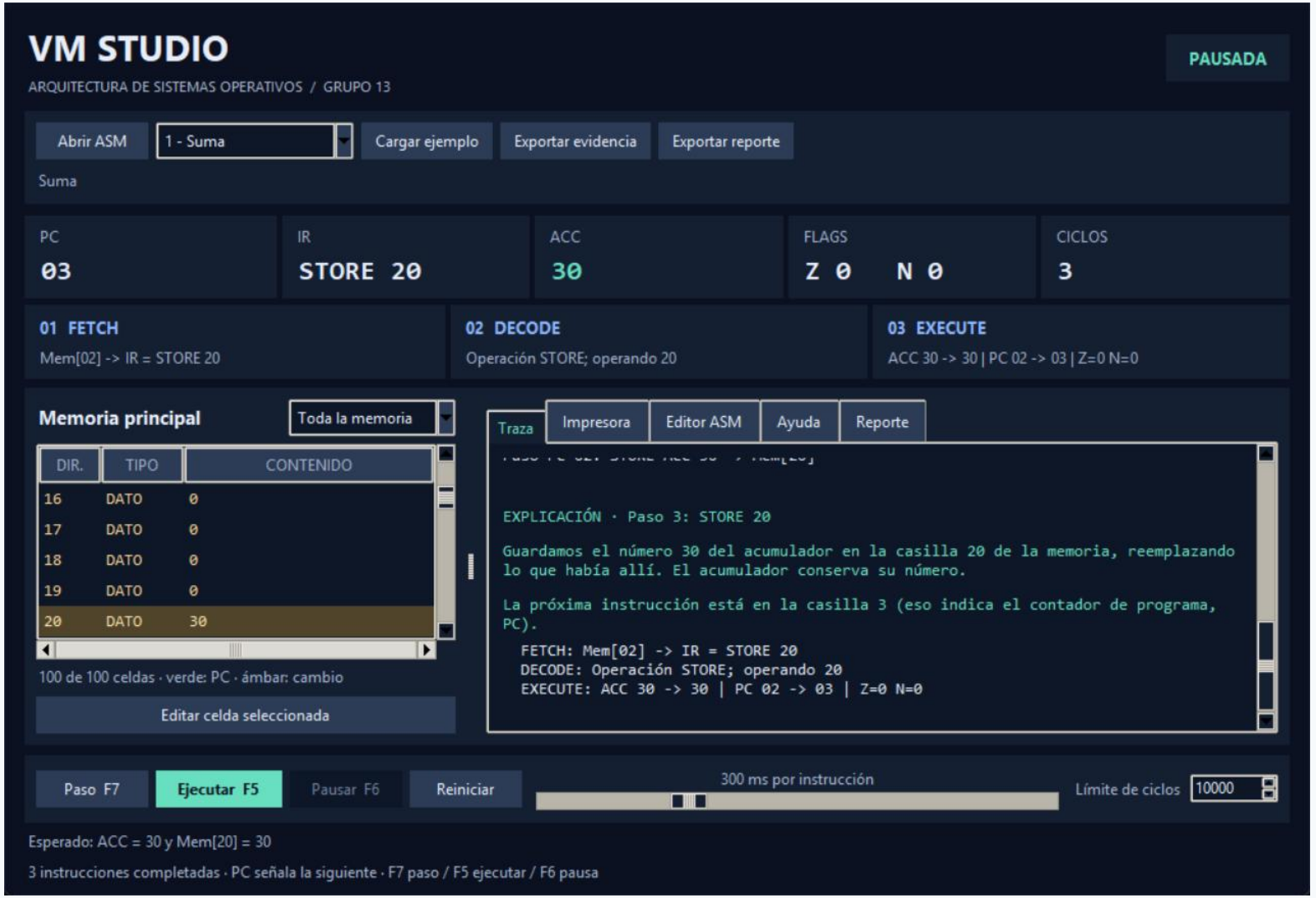
El PC se incrementa después de Decode y antes de Execute.
Un salto puede reemplazar ese destino.

Traza de la suma

Instrucción	PC final	ACC final	Mem[20]
LOAD 10	1	10	0
ADD 11	2	30	0
STORE 20	3	30	30
OUTPUT	4	30	30
HALT	5	30	30

Datos iniciales: Mem[10] = 10 y Mem[11] = 20 · Salida: 30

Demostración de la interfaz



Saltos y bucles

JZ: salto condicional

```
20 – 20 = 0
Z = 1
JZ 6 toma el salto
Salida: 111
```

JNZ: repetición

```
Contador: 3, 2, 1, 0
Repetir mientras Z = 0
Acumular 3 + 2 + 1
Salida: 6
```

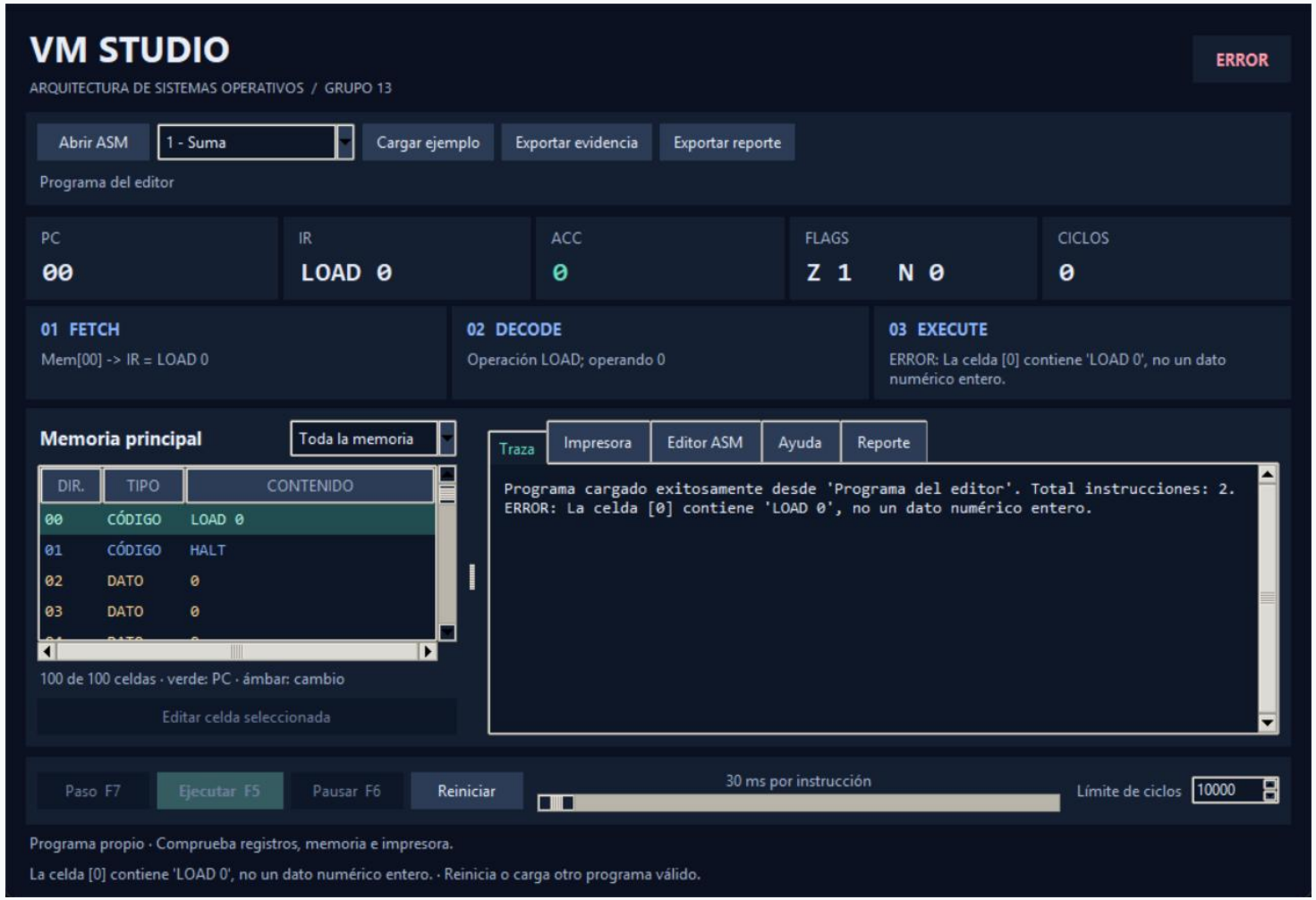
Tener HALT en el archivo no garantiza alcanzarlo: el límite protege la ejecución.

Dispositivos virtuales

Controlador	Interacción	Resultado observable
Teclado	INPUT recibe un entero	ACC = 7
Disco	Leer y analizar un archivo ASM	Programa instalado en memoria
Impresora	OUTPUT emite ACC	Salida virtual: [7]

Cancelar INPUT conserva la instrucción pendiente y permite reintentar sin sumar un ciclo.

Manejo de errores



Durante la carga

MULT 10
Carga rechazada.
Se conserva lo anterior.

Durante la ejecución

LOAD 0 lee código
como un número.
ERROR · PC = 0

Reportes de ejecución



HTML

Consultar e imprimir sin conexión

CSV

Revisar registros y pasos en Excel

JSON

Conservar la evidencia estructurada completa

Pruebas y resultados

Caso	Esperado y obtenido	Estado
Suma	ACC = 30; Mem[20] = 30	Correcto
Resta	ACC = 20; Mem[20] = 20	Correcto
Almacenamiento	Mem[20] = 30	Correcto
Condicional	Salida = 111	Correcto
Bucle	ACC = 6; Mem[53] = 6	Correcto
Entrada y salida	Entrada 7 → salida 7	Correcto

39 pruebas automatizadas correctas · 9 errores de carga comprobados · GUI verificada

Organización del código

```
src/vm_studio/
  motor.py
  interfaz.py
  consola.py
  reportes.py
```

Motor: instrucciones y estado
Interfaz: interacción visual
Consola: terminal y archivos
Reportes: exportación de evidencia

`examples/` `tests/` `docs/` `output/`

Conclusiones y límites

Cada resultado conserva el recorrido que lo explica.

Memoria y registros observables
Ejecución por pasos y automática
Pruebas reproducibles y reportes

Alcance didáctico: 100 celdas y 10 instrucciones.
No simula concurrencia, interrupciones ni una CPU física.